University *of Ljubljana*
Faculty *of Mathematics and Physics*

Department of Physics

Successive Over-Relaxation for Boundary Value Problems

5. Assignment for Model Analysis 2, 2025/26

Author: Marko Urbanč (28232019)
Professor: Prof. dr. Simon Širca
Advisor: doc. dr. Miha Mihovilovič

Ljubljana, March 2026

Contents

1 Introduction **1**

2 Task **2**

2.1 Steady-State Velocity Profile and Poisseuille Coefficient 2

2.2 Steady-State Temperature Distribution in a Cylinder 2

3 Solution Overview **3**

4 Results **6**

5 Conclusion and Comments **6**

1 Introduction

This week we finally move on from shooting methods to a more general class of methods for solving boundary value problems: relaxation methods. Specifically we will be taking a look at the Successive Over-Relaxation (SOR) [1] method, which is an iterative method for solving linear systems of equations that arise from discretizing partial differential equations (PDEs). The SOR method is an extension of the Gauss-Seidel method, which itself is an improvement over the Jacobi method. The key idea behind SOR is to introduce a relaxation factor that can accelerate the convergence of the iterative process. That is to say, that instead of simply using the new values computed in the current iteration, we intentionally "over-relax" the update by a factor ω (the relaxation factor), which can be tuned to achieve faster convergence.

The relaxation factor ω in bounded between 0 and 2, where $\omega = 1$ corresponds to the Gauss-Seidel method, $\omega < 1$ corresponds to under-relaxation, and $\omega > 1$ corresponds to over-relaxation. $\omega = 2$ corresponds to the limit of stability, and values of $\omega \geq 2$ will lead to divergence. The optimal choice of ω depends on the specific problem being solved, and finding it can significantly reduce the number of iterations required for convergence. Specifically, ω depends on the spectral radius of the iteration matrix ρ_{jac} as:

$$\omega = \frac{2}{1 + \sqrt{1 - \rho_{\text{jac}}^2}} \,. \tag{1}$$

The optimal ω for a full square grid can be approximated as:

$$\omega_{\text{opt}} \approx \frac{2}{1 + \frac{\pi}{N}} \,, \tag{2}$$

where N is the number of grid points in one dimension. In our assignment we will use the following ansatz for the relaxation factor:

$$\omega = \frac{2}{1 + \alpha \frac{\pi}{N}} \,, \tag{3}$$

where α is a tunable parameter that we will need to optimize for our specific problem.

Since the SOR method iterative process is symmetric with respect to grid point parity, we can perform updates in a checkerboard pattern in two half-steps. This allows us to use an even faster approach for convergence called **Chebyshev Acceleration**, which is a technique that optimizes the relaxation factor ω over the course of half-iteration steps as:

$$\omega^{(}0) = 1\,, \tag{4}$$

$$\omega^{(1/2)} = \frac{1}{1 - \frac{1}{2}\rho_{\text{jac}}^2}\,, \tag{5}$$

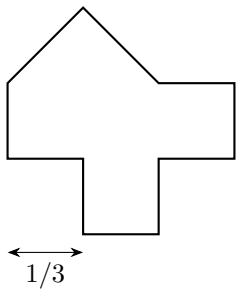
$$\omega^{(n)+1/2} = \frac{1}{1 - \frac{1}{4}\omega^{(n)}\rho_{\text{jac}}^2}\,. \tag{6}$$

This approach allows us to achieve convergence in a number of iterations that scales as $\mathcal{O}(N)$, which is a significant improvement over the $\mathcal{O}(N^2)$ scaling of the standard SOR method. For this to work however we need to have a good estimate of the spectral radius ρ_{jac}, which we can obtain by tuning α in Equation (3) and using the optimal ω in Equation (1) to get ρ_{jac}.

2 Task

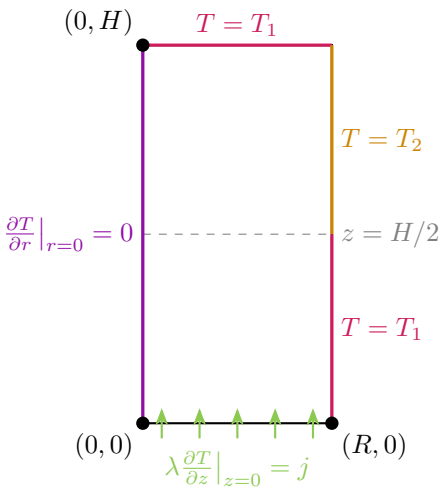
2.1 Steady-State Velocity Profile and Poisseuille Coefficient

Using the SOR method, we will solve the Poisson equation for the velocity profile of a fluid flowing through a pipe with a *'house-like'* cross-section. We are instructed to pay close attention to the correct treatment of cases where the boundary conditions are not defined at the grid points, but are instead defined at the midpoints between grid points. We need to find the optimal parameter α for the relaxation factor ω and test out the Chebyshev acceleration method. Finally, we need to compute the Poisseuille coefficient for the flow.

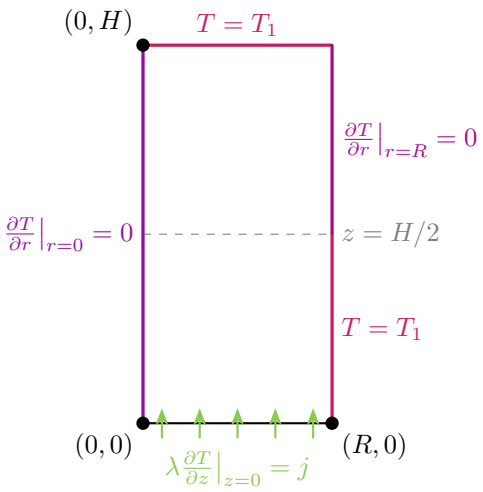


2.2 Steady-State Temperature Distribution in a Cylinder

Next, we will solve the steady-state heat equation in an equilateral cylindrical geometry with a constant heat flux at the bottom of the cylinder. The cylindrical surface is cut horizontally in half, with each half at a different temperature. The top surface of the cylinder is kept at the same temperature as the lower half of the cylinder. Our goal is to find the steady-state temperature distribution. We need to then repeat this calculation for the same geometry, but where the upper half of the cylinder is thermally insulated.



(a) Case 1: **Dirichlet** boundary condition at the upper half of the cylinder.



(b) Case 2: **Neumann** boundary condition at the upper half of the cylinder.

3 Solution Overview

Ha! This week was perhaps more involved than the previous ones. I guess you could say that I was up to my *old shenanigans* again. This week the standard Python computational stack of `numpy`, `scipy`, and `matplotlib` was augmented by a favorite of mine `mpi4py`, which provides bindings for the Message Passing Interface (MPI) standard in Python.

As stated in the introduction, the SOR method is symmetric with respect to grid point parity, which means we can independently update the two bipartite sublattices of the grid in a checkerboard pattern. This makes parallelization of the SOR method via domain decomposition very straightforward. We can divide the full lattice into smaller subdomains, which I will refer to as "chunks", partially inspired by that one block game. Each chunk can be assigned to a different MPI rank, and the updates can be performed in parallel. The only communication that is required between the ranks is the exchange of boundary values between neighboring chunks after each half-step of the SOR iteration, but since we can independently update the two sublattices *ghost halo* broadcasting very simple and efficient.

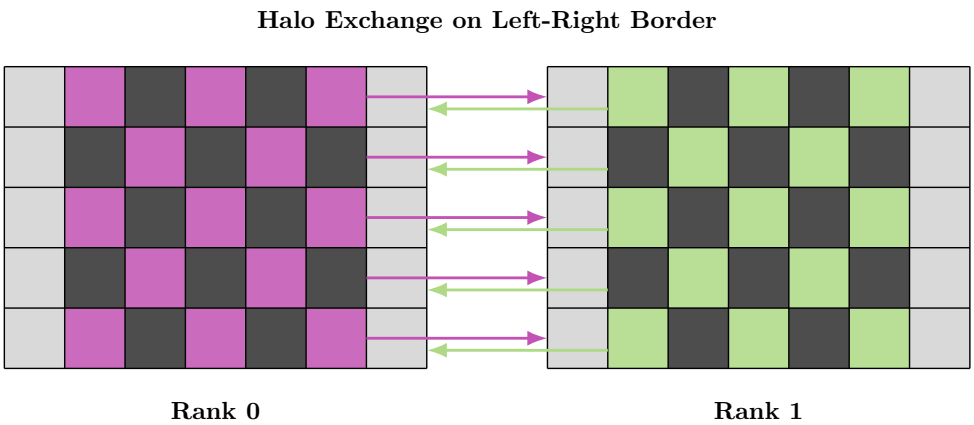


Figure 2: Ranks with Purple-Black and Green-Black sub-lattices exchanging a ghost column on their shared border.

I'll skip the various details of the implementation, otherwise I'll never beat the *too much comp-sci* allegations. But I will say that the implementation was very stable even with differing rank topologies and sublattice sizes. Overriding the base Cartesian class to implement the cylindrical Laplace was also quite simple. As for boundary point handling, I used the well known **Shortley-Weller** stencil [2], which is locally a first-order discretization, but globally second-order accurate [3]. The weights of the stencil are computed as:

$$w_R = \frac{1}{\theta_R(\theta_R + \theta_L)}\,, \tag{7}$$

$$w_L = \frac{1}{\theta_L(\theta_R + \theta_L)}\,, \tag{8}$$

$$w_U = \frac{1}{\theta_U(\theta_U + \theta_D)}\,, \tag{9}$$

$$w_D = \frac{1}{\theta_D(\theta_U + \theta_D)}\,, \tag{10}$$

where θ_R, θ_L, θ_U, and θ_D are the fractional distances from the grid point to the boundary in the right, left, up, and down directions respectively. I computed fractional distances once at the beginning of the simulation by ray-casting from each boundary-adjacent point in each of the four directions to the boundary against the boundary contour that was extracted via the marching squares algorithm. The update at the boundary-adjacent interior point is then computed as a weighted average as:

$$\tilde{u}_i = \frac{w_R u_R + w_L u_L + w_U u_U + w_D u_D - q_i \Delta x^2}{w_R + w_L + w_U + w_D}\,, \tag{11}$$

such that the SOR update is then given by:

$$u_i^{(n+1)} = u_i^{(n)} + \omega \left(\tilde{u}_i - u_i^{(n)} \right) \tag{12}$$

I also found recent development in higher order stencils exactly for in solvers that use ghost halo exchange [4], however this is entirely unnecessary for our purposes. In fact, I found that the pure 5-point stencil (equivalent to Shortley-Weller with $\theta = 1$) is already sufficiently accurate for our purposes of computing the Poisseuille coefficient on the house-shaped domain. At $N \approx 400$ the problematic areas (diagonals and 270° corner points) are few in comparison to the rest of the domain, and the error is localized to those areas. The local area error is then of order $\mathcal{O}(\Delta x)$, but the global error, an integral over the whole domain, is of order $\mathcal{O}(\Delta x^2)$, which likely got *eaten* by my generous tolerance of 10^{-6} for convergence.

Before moving on the main results we can attempt to verify that our implementation works properly by applying it to test cases for which we know the analytical solution. We can start with the *trivial* case of a square domain with Dirichlet boundary conditions. We know that the Poisseuille coefficient for a unit square domain is $C_{\text{square}} = 0.0351443$ [5, Section 3-3]. We can sweep over α to find the optimal ω for which the number of iterations required for convergence is minimized, and also observe how the Poisseuille coefficient changes in that process. Figure 3 shows the results of the scan over α for the square case.

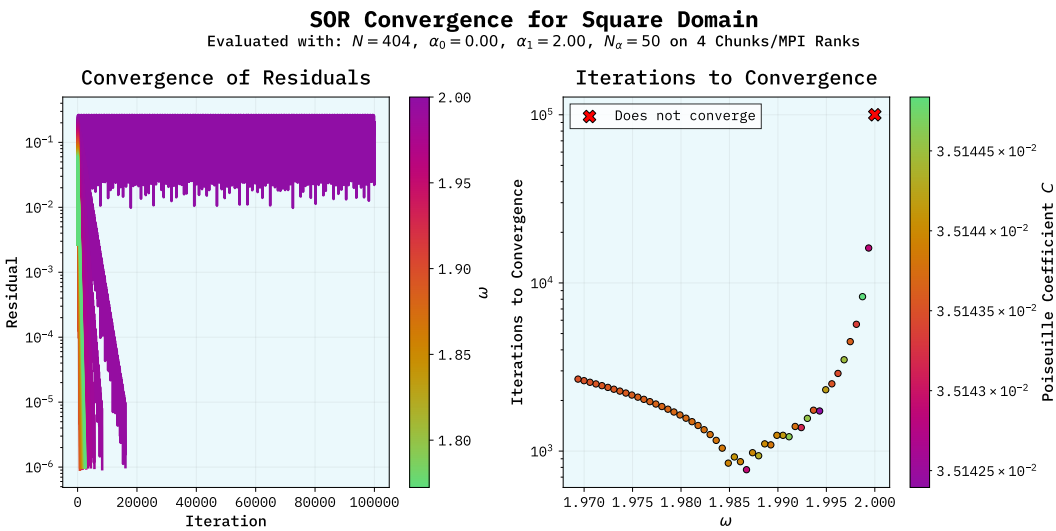


Figure 3: Scan over α for the square domain. The left-hand plot shows the convergence of the residuals for different values of α, while the right-hand plot shows the number of iterations required for convergence and the computed Poisseuille coefficient for each value of α.

I intentionally included the point $\alpha = 0.0$ in the scan, which corresponds to $\omega = 2.0$, to show that the method never converges, as is evident by the residuals not decreasing at all. Taking a more zoomed in look we see that the optimal value of ω is around 1.985, which for $N = 404$ corresponds to $\alpha \approx 0.98$, which is very close to the optimal value of $\alpha = 1.0$ that we would get from Equation (2). As seen from the colors on the right-hand scatter plot, we can see that the Poisseuille coefficient is very close to the analytical value for all values of $\alpha \in [0.0, 2.0]$, which were tested in this scan. So with this we've seen that our solver converges to the correct solution and that it does so at the optimal ω that is close to the theoretical prediction.

Next we can solve the Poisson equation for flow in a circular pipe. A classical result for the Poisseuille coefficient for a circular pipe is $C_{\text{circle}} = R^2/32 \approx 0.03125$ [6], where $R = 1$ is the radius of the pipe. Figure 4 shows the results of a wide scan over α for the circular pipe. It is clear that such wide scans are not very useful for finding the optimal α since they are too coarse, but they do show the general behavior of the convergence with respect to α.

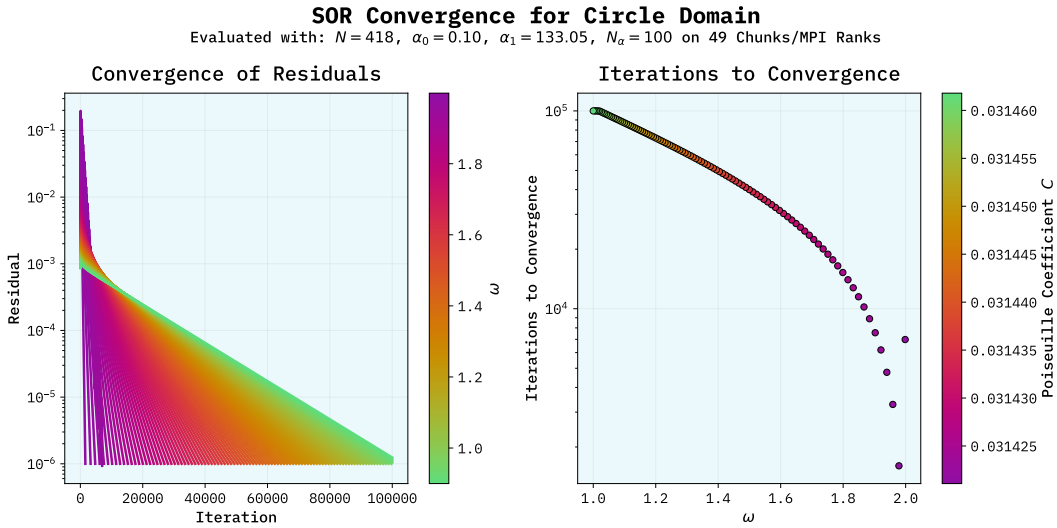


Figure 4

We can clearly see that there exists an optimal ω (and thus α) for which the number of iterations required for convergence is minimized, however since our scan was wide and coarse we cannot determine the exact value from this plot. Figure 5 shows a zoomed in version of the scan around the optimal α, where we can read that the optimal ω is around 1.985, which for $N = 404$ corresponds to $\alpha \approx 0.98$, which is the same optimal ω as for the square case, and is again very close to the optimal value of $\alpha = 1.0$ that we would get from Equation (2). This a sensible result if we consider that the optimal ω is determined by the spectral radius of the iteration matrix. The spectral radius is determined by the largest eigenvalue of the iteration matrix, which is tied to the *characteristic length* of the domain, **not** its shape. Since the characteristic length of the circular pipe is its radius, which is the same as the side length of the square domain, it makes sense that the optimal ω is the same for both cases.

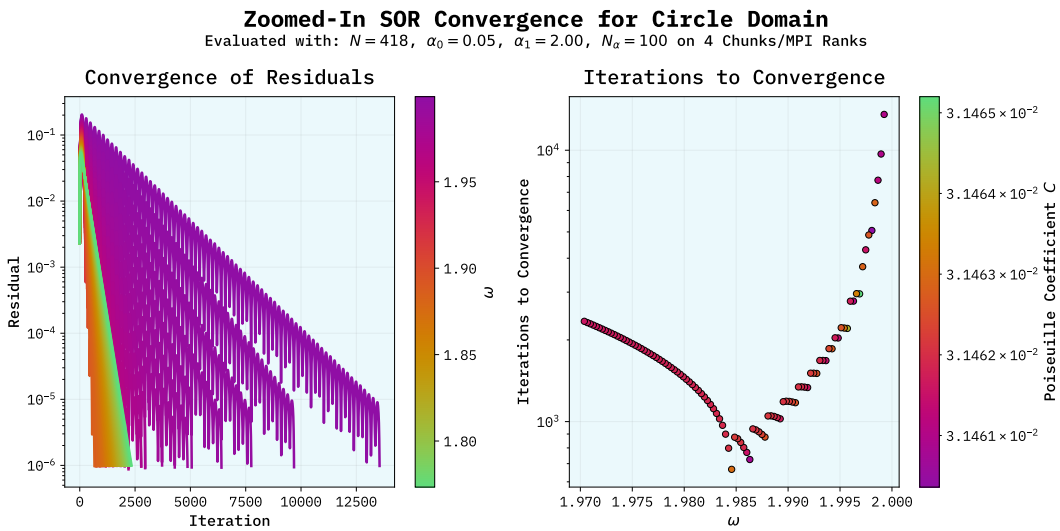


Figure 5

As for the Poisseuille coefficient, we can see on the wider scan that the values do change with α, but they are all close regardless of the value of α. Upon closer inspection for $\alpha \in [0.05, 2]$ we can see that the computed Poisseuille coefficient is very close to the analytical value, reaching $C_{\text{circle}}^{(\text{num})} = 0.03146$ at the optimal α. With that I hope we've built a compelling case for the correctness of our implementation, and we can now move on to the main task of computing the velocity profile and Poisseuille coefficient for the house-shaped domain.

4 Results

5 Conclusion and Comments

References

[1] David Young. "Iterative Methods for Solving Partial Difference Equations of Elliptic Type". In: *Transactions of the American Mathematical Society* 76.1 (1954), pp. 92–111. ISSN: 00029947, 10886850.

[2] G. H. Shortley and R. Weller. "The Numerical Solution of Laplace's Equation". In: *Journal of Applied Physics* 9.5 (May 1938), pp. 334–348. ISSN: 0021-8979. DOI: 10.1063/1.1710426.

[3] J. H. Bramble and B. E. Hubbard. "On a Finite Difference Analogue of an Elliptic Boundary Problem which is Neither Diagonally Dominant Nor of Non-negative Type". In: *Journal of Mathematics and Physics* 43.1-4 (1964), pp. 117–132. DOI: https://doi.org/10.1002/sapm1964431117.

[4] Armando Coco and Giovanni Russo. *High Order Finite-Difference Ghost-Point Methods for Elliptic Problems in Domains with Curved Boundaries*. 2024. DOI: 10.48550/arXiv.2405.13986.

[5] Frank M. White. *Viscous Fluid Flow*. 3rd ed. Section 3-3, p. 106 Poiseuille Flow through Ducts. New York: McGraw-Hill, 2006. ISBN: 978-0-07-240231-5.

[6] L.D. Landau and E.M. Lifshitz. "Chapter II - Viscous Fluids". In: *Fluid Mechanics (Second Edition)*. Ed. by L.D. Landau and E.M. Lifshitz. Second Edition. Pergamon, 1987, pp. 44–94. ISBN: 978-0-08-033933-7. DOI: https://doi.org/10.1016/B978-0-08-033933-7.50010-6.